

JSON OUTPUT & PARSING CHEATSHEET

AI Engineer Ready · Week 3 — Turning Model Text Into Structured Data

Golden Rule → ask for JSON with an example → print raw text → parse inside try/except → validate keys

1. Why Ask a Model for JSON?

Free text is fine for humans; code needs **structured data**. Asking the model to respond in JSON lets you turn its answer straight into a Python `dict` / `list` you can use in the rest of your app.

2. Step 1 — Ask for JSON Explicitly (With an Example)

```
few_shot_template = '''Return a JSON object with keys 'summary' and 'language'.
```

Example:

Input: I love programming in Python.

Output: `{{"summary": "likes programming", "language": "English"}}`

Now respond to:

Input: `{text}`

Output: '''

```
query = few_shot_template.format(text="Bonjour, j'aime coder en Python.")
```

```
resp = chat.invoke(query)
```

```
raw_text = resp.content
```

Note the `{{ { }}` — inside a `.format()` template, a literal brace must be doubled or Python treats it as a placeholder.

3. Step 2 — Always Print the Raw Response First

```
print("--- Raw LLM response ---")
```

```
print(raw_text)
```

Non-negotiable. If parsing fails, the raw text is the only way to see why (extra commentary, markdown fences, trailing commas...).

4. Step 3 — Parse Inside try/except

```
import json
```

```
try:
```

```
    parsed = json.loads(raw_text)
```

```
    print(parsed["summary"])
```

```
except json.JSONDecodeError as e:
```

```
    print("Could not parse JSON:", e)
```

5. Common Failure Modes & Fixes

SYMPTOM	CAUSE	FIX
Expecting value	Model added text before/after JSON	"Respond with ONLY valid JSON, no extra text."
<code>JSONDecodeError</code> on trailing comma	Model mimicked "human-friendly" JSON	Ask for strict JSON; retry once on failure

SYMPTOM	CAUSE	FIX
Wrapped in ```json fences	Model defaults to markdown	Strip fences before parsing, or say "no code fences"
KeyError after parsing	JSON parsed but missing expected key	Validate keys exist before using them
Works sometimes, fails other times	Temperature too high	Lower temperature (e.g. 0) for structured tasks

6. Defensive Parsing Pattern (Recommended)

```
def safe_parse_json(raw_text: str):
    try:
        return json.loads(raw_text), None
    except json.JSONDecodeError as e:
        return None, str(e)

parsed, error = safe_parse_json(raw_text)
if error:
    print("Parse failed:", error, "| Raw:", raw_text)
```

Checklist

- ☐ Prompt shows an example of the exact JSON shape wanted
- ☐ Prompt says "ONLY valid JSON" / "no extra text"
- ☐ Raw response printed before parsing (while debugging)
- ☐ `json.loads()` wrapped in try/except
- ☐ Keys validated before use downstream